AI Cart V2

Custom racing-car controller trained with evolutionary mutation and saved as a .npz model.

Agent script: Albrain_Vedant.py

Saved model: Albrain_Vedant.npz

Goal

Drive farther and survive longer by balancing speed, steering, and wall avoidance.

Vedant Makkar

Reinforcement learning challenge

What the Project Does

Engine

Pygame simulation runs the car, track, sensors, physics, collisions, training, saves, and benchmarks.

Student agent

My file defines a brain class that receives sensor data and returns four driving actions every frame.

Training result

The best parameters are saved and reloaded from an .npz file for future benchmark runs.

```
class AIbrain_Vedant:
    def decide(self, data):
        return [throttle, brake, left, right]

    def mutate(self):
        # create next generation

    def get_parameters(self):
        return copy.deepcopy(self.parameters)
```

- I focused on making the agent stable first, then using evolution to tune it.
- The main challenge was keeping speed high without getting stuck in sharp corners.

Inputs and Outputs

The decision loop turns sensor readings into driving commands.

```
N_RAYS = 9
N_INPUTS = N_RAYS + 1   # 9 rays + speed
N_HIDDEN = 12
N_ACTIONS = 4

rays_norm = np.clip(rays / RAY_NORM, 0.0, 1.0)
speed_frac = np.clip(self.speed / SPEED_NORM, 0.0, 1.0)

x[:N_RAYS] = rays_norm
x[N_RAYS] = speed_frac
```

Inputs

Nine ray sensors measure how much space is around the car. Speed is added so the agent knows when it is moving too fast for the available clearance.

Outputs

The four action values represent throttle, brake, left, and right. The agent uses sigmoid outputs so every command stays between 0 and 1.

Why normalize?

Scaling rays and speed into similar ranges prevents one measurement from dominating the decision.

How decide() Chooses an Action

```
front = rays[4]
left_space  = dot(rays[5:], left_weights)
right_space = dot(rays[:4], right_weights)

base_logits = heuristic_logits(rays, feats)
hidden = tanh(W1.dot(x) + b1)
residual = W2.dot(hidden) + b2

logits = base_logits + trust * residual
out = sigmoid(logits)
```

Heuristic layer

This is the “common sense” driver: accelerate when front space is clear, brake when speed is risky, and steer toward open space.

Neural residual

A small neural network adds corrections. The trust vector controls how much the neural output can change each action.

Reason for this design

Pure random weights were unstable. A hybrid approach gave the car a baseline strategy while still allowing evolution to improve it.

Corner and Safety Logic

Commit direction

When the front ray becomes short, the agent commits to one steering direction for a few frames. This reduces left-right flicker at tight turns.

Stuck recovery

The agent tracks recent movement. If it barely moves while facing a corner, it flips steering temporarily to escape.

Safe speed

Speed is limited based on front clearance. The closer the wall, the faster throttle cuts out and braking turns on.

```
in_corner = front < corner_front_thresh

if in_corner:
    commit_direction = proposed_direction
    commit_frames_left = commit_frames

if stuck_frames > 12:
    escape_direction = -steer_direction
    escape_hold = 15

if speed_frac > safe_speed_frac * front:
    out[0] = 0.0      # throttle off
    out[1] = 1.0      # brake if too fast
```

Training: Evolution Instead of Backpropagation

```
mutate_rate = 0.4

h_sigma = decayed_sigma("heuristic", 0.25, 0.02)
for key in self.heuristic:
    if random() < mutate_rate:
        self.heuristic[key] += noise

# mutate neural weights and biases
W1, b1, W2, b2, trust = mutated_versions

score = distance + 0.1 * (distance / time)
```

What evolves

The algorithm mutates heuristic values, neural weights, biases, and the trust vector.

Why decay mutation size

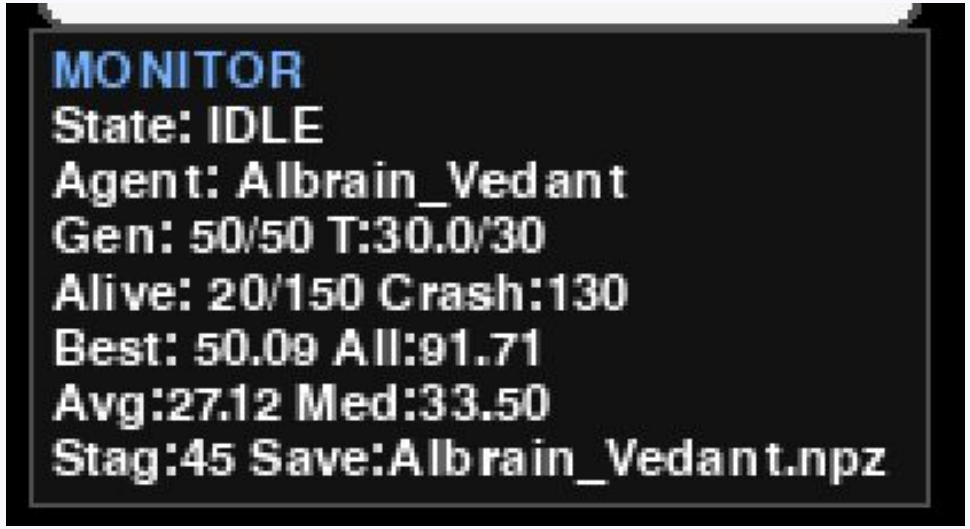
Large early mutations explore different strategies. Smaller later mutations fine-tune the best behavior.

How score works

The score rewards distance, with a small bonus for average speed. This encourages progress without only focusing on speed.

Final Training Snapshot

The final run completed 50/50 generations and saved Albrain_Vedant.npz.



Final stats from monitor

Gen: 50/50
Alive: 20/150
Best current: 50.00
Best all-time: 91.71
Average: 27.12

Main takeaway

The best version was not a huge neural network. The working solution was a simple, explainable driver with evolutionary tuning around the important parameters.

Next step: run the saved .npz in benchmark mode and compare against classmates' agents.